Ira A. Fulton Schools of Engineering
Arizona State University

CSE 531: Distributed and Multiprocessor Operating Systems

Logical Clock Written Report

Problem Statement

The gRPC banking system contains multiple customer and branch processes that exchange deposit, withdrawal, query, and propagation messages. Because the processes run independently, physical time cannot reliably establish the order in which distributed events occurred. The problem is to extend the completed gRPC banking project so that every customer and branch maintains a Lamport logical clock and records timestamps that preserve the happens-before relationships among related events.

Each customer deposit or withdrawal must be received by one branch, applied to that branch's balance, and propagated to every other branch. The resulting event log must show the correct causal order for the customer send, branch receive, propagation sends, and propagation receives.

Goal

The goal is to implement Lamport's logical clock algorithm in the gRPC banking system and produce a verifiable JSON event log. The implementation must ensure that a causally later event always has a greater logical-clock value than the event that caused it.

For each customer deposit or withdrawal, the program records the customer send event, the receiving branch event, every branch propagation send, and every receiving branch event. The final output.json contains customer logs, branch logs, and a combined per-request event log so the ordering can be checked.

Setup

The project was implemented in Python using gRPC for communication between independently running customer and branch processes. GitHub was used for source control and project organization.

Required libraries and versions were:

- Python

- grpcio==1.71.2
- grpcio-tools==1.71.2
- protobuf==5.29.6

The protobuf definition file, *protos/banks.proto*, defines the Request and Response messages and the MsgDelivery RPC. The request message includes the customer request ID, operation interface, transaction amount, Lamport logical-clock value, and sending branch ID. Python gRPC stub files were generated from the protobuf definition.

The project is run in two terminals:
- python server.py input.json
- python client.py input.json

The server starts one branch process for each branch in the input. The client waits for the branch servers, sends the customer requests, collects the logs, writes output.json, and signals the servers to stop.

Implementation Processes

The implementation adds an independent logical_clock to every customer and branch process. Each clock begins at 0 and belongs only to its own process. I extended the protobuf request message with logical_clock and sender_branch_id fields. The logical-clock field carries a Lamport timestamp with each gRPC message, while the sender-branch field identifies the branch that originated an inter-branch propagation message.

In customer.py, the customer increments its logical clock before sending each request and attaches that timestamp to the gRPC request. Deposit and withdrawal requests are recorded in the customer event log with the customer request ID, logical-clock value, interface name, and event description.

In branch.py, each branch applies Lamport's receive rule whenever it receives a customer request or an inter-branch propagation message: logical_clock = max(local_clock, received_clock) + 1. This ensures that the receive event has a timestamp greater than both the branch's prior local time and the timestamp included with the received message.

When a branch receives a customer deposit or withdrawal request, it updates its clock using the receive rule, applies the balance update, and records the receive event. It then sends the resulting update to every other branch. Before each propagation send, the branch increments its logical clock again so that every send event has its own distinct timestamp. When a branch receives a propogate_deposit or propogate_withdraw message, it applies the receive rule, updates its balance, and records the receive event. It does not propagate that same update again, preventing duplicate updates and infinite propagation loops.

In client.py, I normalized the logical-clock input format, processed requests in customer-request-id order, collected the event logs from customers and branches, and generated the required output: events for each customer, events for each branch, and a flat event list for deposit and withdrawal requests. I also added documentation, docstrings, comments explaining Lamport's rules, and a README containing setup, execution, and verification instructions. The assignment uses the interface spellings propogate_deposit and propogate_withdraw, so the implementation intentionally retains those names for required-output and autograder compatibility.

Results

The implementation successfully produced a valid distributed event log for four customers and four branches. The final input contained eight customer dedddposit or withdrawal requests. Each request produced one customer send event and seven branch events: one receiving-branch event, three propagation-send events, and three propagation-receive events.

The final results were:

- Customer events: 8
- Branch events: 56
- Total events in the combined request log: 64
- Unit tests passed 11 out of 11
- Gradescrop Logical Clock autograder results: 64 out of 64 correct events

The Gradescope results verified that all eight customer timestamps were correct, all 56 branch events followed the required happens-before relationships, and all 64 combined events were correctly ordered. These results justify that the customer and branch logical clocks were updated correctly and that Lamport's algorithm was successfully applied to the distributed gRPC banking system.